2do Parcial de Arquitectura y Organización del Computador - 2c2025

Normas generales

- El parcial es INDIVIDUAL
- El parcial es a libro abierto, pueden disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada o a cualquier otro recurso que consideran. Sin embargo, **está totalmente prohibido el uso del celular, internet y chatbots.**
- El parcial es en los laboratorios, usando las computadoras de los laboratorios o sus propias computadoras, siempre que sean soportadas por el sistema de parciales detallado más adelante.
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Vamos a usar herramientas de detección de plagio para asegurarnos de que su entrega sea original. La entrega debe ser original para estar aprobada. El plagio en instancias de evaluación incurre sanciones que pueden incluir la suspensión académica.

Respecto al sistema de parciales:

- Deberán conectarse al sistema de parciales de la materia, el cual limitará su acceso a internet para que únicamente se puedan conectar a GitHub. Se debe **mantener la conexión** a este sistema hasta entregar el examen.
- Les proveeremos de un script que deberá ejecutarse al comienzo del parcial y no detenerse hasta haber entregado. Con él nos aseguramos que permanezcan conectados al sistema.
- Para aprobar el parcial deben implementar 2 de los 3 puntos del enunciado y que corran con éxito los tests funcionales, de abi y de memoria (valgrind).

Régimen de Aprobación

En este parcial se evalúa el manejo de los mecanismos de:

- Excepciones, interrupciones de hardware y syscalls mediante Interrupciones al procesador
- Manejo de memoria mediante Segmentación y Paginación
- Permisos y cambios de nivel de privilegio
- Administración del tiempo de procesamiento y del espacio de memoria para la ejecución de tareas.

Para la Arquitectura Intel x86. La entrega debe demostrar, en la medida exigida por el enunciado, comprensión clara de la mayoría estos mecanismos, e incomprensión de ninguno.

El parcial se aprueba con un puntaje numérico de **60 o más**, y califica para régimen de promoción con un puntaje numérico de **80 o más**.

NOTA: Lea el enunciado del parcial hasta el final antes de comenzar a resolverlo.

Régimen de Promoción (solo para quienes estén rindiendo por primera vez con ausencia justificada a la primer fecha)

Quien reciba un puntaje de **80 o más** en este parcial y tenga una calificación suficiente para promoción en el primer parcial podrá acceder al régimen de promoción. Esto es válido únicamente para la primer instancia de los parciales y no para sus recuperatorios.

Modalidad de Entrega

> [!WARNING] Se estará circulando por el aula una planilla escrita donde **deben** anotarse para que tengamos constancia de que su presencia en los laboratorios al momento del parcial. De no anotarse no podremos considerar su entrega como válida. La planilla, luego de circular, se encontrará en los escritorios al frente de cada laboratorio. **No se vayan sin anotarse!**

Deberán crear una nueva branch en este repositorio y allí desarrollar su solución en el archivo `Resolucion.md`, en formato Markdown (Acá tienen un machete de cómo dar formato en markdown). Es importante que no utilicen otro formato ya que dificultaría la corrección.

Si su solución utiliza código modificado del Trabajo Práctico, es importante que lo incluyan además de explicar en palabras las modificaciones (pueden copiar y pegar cualquier cosa de sus repositorios de TP). También pueden incluir imágenes si así lo desean. De tener dudas sobre qué funciones se pueden o no asumir implementadas, preguntar a los docentes durante el examen.

Una vez finalizado, completarán la entrega **creando un Pull Request e incluyendo a ayoc-bot como reviewer. Deben ver un mensaje de "Recibido!" de ayoc-bot para confirmar la entrega.**

Contexto

Partiendo de un sistema como el del TP, nos piden que lo extendamos para poder controlar el proceso de producción de monitores en una fábrica automatizada. En términos generales, las máquinas de la fábrica consumen unos recursos y producen otros. Por ejemplo, la máquina que produce el monitor terminado requiere: el panel led, la estructura, la carcasa, el chip controlador, etc. Todos estos recursos que necesita son producidos por otras máquinas, que a su vez pueden requerir recursos producidos por otras.

Para administrar este flujo de recursos, cada máquina estará controlada por una tarea que correrá sobre nuestro sistema. Las tareas en sí serán desarrolladas por otro equipo con el que acordamos algunas cuestiones:

- El kernel proveerá una **syscall** `solicitar` que permita a la tarea llamadora solicitar un recurso (informando el tipo de recurso que busca). **No se devolverá el control** a la tarea solicitante hasta que el recurso esté listo.
- El kernel proveerá una **syscall** `recurso_listo` que permita a la tarea llamadora avisar que su recurso está listo. La misma habrá dejado información en los 4KB a partir de la dirección virtual `0x0AAAA000` que **deberá ser copiada** a los 4KB a partir de la dirección virtual `0x0BBBB000` de la tarea que pidió el recurso (las direcciones podrían no estar mapeadas, en cuyo caso habría que **limpiarlas antes de acceder**). Una vez informado que el recurso está listo y copiada la información, la tarea llamadora debe ser desalojada y restaurada por completo a su estado inicial.
- El kernel proveerá un mecanismo de arranque manual mediante una **interrupción externa** producida al presionar un botón, a la que le corresponde la entrada

41 de la IDT. Su efecto será iniciar la producción del `recurso_t` indicado en la dirección `0xFAFAFA` del espacio del kernel, bajo la misma lógica que `solicitar`.

Aclaraciones:

- Toda tarea es productora y potencial solicitante.
- Para todo recurso existe una tarea que lo produzca.
- La fábrica está dispuesta de tal forma que no existen casos en los que se forman cadenas infinitas de dependencias (e.g. productora de A solicita recurso B que solicita recurso C que solicita recurso A).
- Pueden asumir que las tareas ya fueron inicializadas.

Ejercicio 1 (50 puntos):

Detallar todas las modificaciones necesarias para que el kernel provea las sycalls `solicitar` y `recurso_listo`. Esto incluye pero **no se limita** a:

- Establecer el mecanismo por el cual las tareas pasarán parámetros a las syscalls.
- Modificar las estructuras necesarias para que las syscalls estén disponibles.
- Establecer el mecanismo por el cual la tarea solicitante no retoma su ejecución hasta tener listo el recurso pedido. (Tener presente que durante la ejecución de una syscall, las interrupciones están deshabilitadas).
- Establecer el mecanismo por el cual se copia la información de la tarea productora a la tarea solicitante.

Cuentan con las siguientes **funciones ya implementadas**, para este ejercicio no hace falta que consideren cómo están implementadas. Su uso es alentado pero no es obligatorio:

- `c task_id_t hay_tarea_disponible_para_recurso(recurso_t recurso)`

```
Dado un tipo de recurso, retorna el id de la tarea disponible para producir el recurso si es que hay, 0 si todas las tareas que proc
```

```
task_id_t para_quien_produce(task_id_t id_tarea)
```

Dado un id de tarea productora, devuelve el id de la tarea que solicitó la producción. Si la producción se solicitó mediante el arranque manual, devuelve 0. Si id_tarea corresponde a una tarea que no está produciendo, el comportamiento de la función es indefinido.

```
void restaurar_tarea(task_id_t id_tarea)
```

Dado un id de tarea, realiza las modificaciones necesarias para que la próxima vez que sea ejecutada empiece desde el comienzo y sin arrastrar cambios de su ejecución anterior.

```
task_id_t hay_consumidora_esperando(recurso_t recurso)
```

Dado un tipo de recurso, retorna el id de una tarea que esté esperando que se disponibilice una productora del mismo. De no haber, retorna 0.

Nota: `task_id_t` es un renombre de `int8_t`.

Ejercicio 2 (25 puntos):

Implementar `restaurar_tarea`. La tarea debe quedar en condiciones para que cuando retome su ejecución, lo haga como si no se hubiese ejecutado antes. Pueden asumir que en su ejecución anterior no utilizó páginas nuevas aparte de `0x0AAAA000` y `0x0BBBB000`.

Considerar con detenimiento y detallar en qué punto de la ejecución del sistema se ejecuta esta función.

Ejercicio 3 (15 puntos):

Detallar todas las modificaciones necesarias para que el kernel provea el arranque manual mediante interrupción externa.

Se pueden asumir implementadas las funciones dadas en el ejercicio 1.

Ejercicio 4 (10 puntos):

Suponiendo que conocen de antemano qué recurso produce cada tarea, asumiendo que hay a lo sumo 10 tareas, y que cada una produce un único recurso, pero puede haber más de una que lo produzca. Considerar:

- ¿Cómo modificarían el sistema para almacenar qué recurso produce cada tarea?
- ¿Cómo modificarían el sistema para llevar registro de qué tareas están esperando a que se liberen productoras de su recurso solicitado?

De las funciones dadas, implementar aquellas que se hayan utilizado en la solución de los ejercicios anteriores, considerando sus respuestas a los subitems previos.

A tener en cuenta para la entrega

- Se parte de un sistema igual al del TP-SP terminado, sin ninguna funcionalidad adicional.
- Indicar todas las estructuras de sistema que deben ser modificadas para implementar las soluciones.
- Está permitido utilizar las funciones desarrolladas en las guías y TPs, siempre y cuando no utilicen recursos con los que el sistema no cuenta.
- Es necesario que se incluya una **explicación con sus palabras** de la idea general de las soluciones.
- Es necesario explicitar todas las asunciones que hagan sobre el sistema.

- Es necesaria la entrega de **código que implemente las soluciones**.
- `zero_page()` modifica páginas basándose en una **dirección virtual** (no física!)